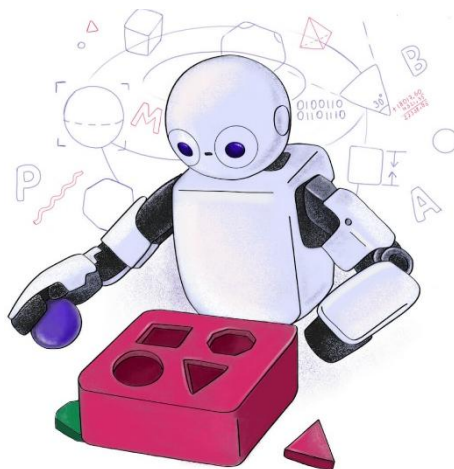


C24 - Inteligência Artificial: *Transformações dos dados*



DADOS BAGUNÇADOS: O caos que você conhece



DADOS PRONTOS PARA O MODELO: Música para os ouvidos da IA!



A jornada de todo cientista de dados: da bagunça à beleza

Continuamos na
jornada de
curadoria dos
dados.

O que veremos?

- Veremos algumas técnicas de preparação e transformação (ou pré-processamento) dos dados:
 - Divisão do *dataset*,
 - Escalonamento de atributos,
 - Codificação de atributos,
 - Engenharia de atributos,
 - Extração de atributos,
 - Seleção de atributos.

Antes de falarmos sobre as transformações, temos que falar sobre a divisão do *dataset* original



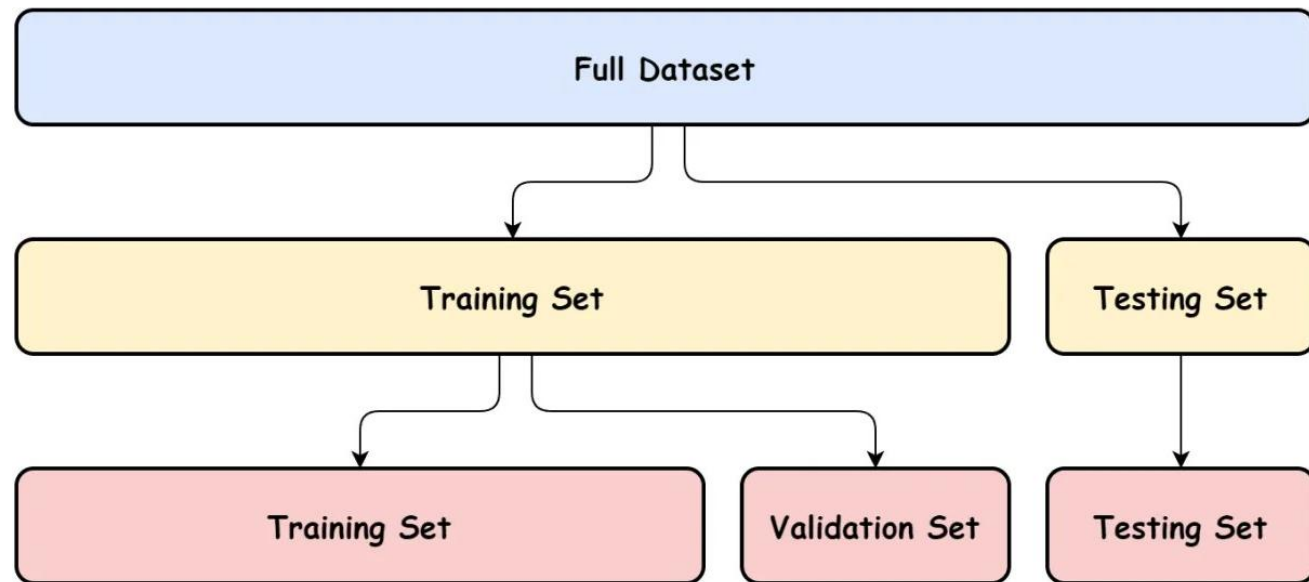
Não é correto usar o mesmo conjunto de dados para treinamento e avaliação dos modelos!



Por que não usar os mesmos dados para treinamento e teste

- Avaliar um modelo com os mesmos dados usados no treinamento **não mede sua capacidade real de generalização**.
- O resultado pode ser bastante **otimista**, pois o modelo é avaliado nos mesmos dados utilizados para seu ajuste.
 - Se o modelo **memorizar os dados de treinamento** (*overfitting*) em vez de aprender padrões gerais, o resultado da avaliação será excelente, mas o modelo pode não generalizar bem.
- Além disso, se o modelo for **otimizado** com base no desempenho no próprio conjunto de treinamento, pode ocorrer *overfitting* aos dados de treinamento.
 - Os hiperparâmetros do modelo podem ser ajustados excessivamente às características específicas do conjunto de treinamento, **comprometendo sua capacidade de generalização**.

Divisão do *dataset*



- Como em ML, o **objetivo é encontrar modelos que generalizem** bem para dados novos (i.e., **inéditos**), precisamos definir uma **forma de avaliar a generalização**.
- Para isso, o conjunto total de amostras (*dataset*) é **dividido em três subconjuntos independentes**:
 - Treinamento
 - Validação
 - Teste

Divisão do *dataset*

- Todos os três devem ser **representativos do problema ou fenômeno** sendo analisado.
 - **Representativos:** O *dataset* completo e, conseqüentemente, os subconjuntos devem **refletir a mesma distribuição de dados do problema**.
 - **Exemplo:** Se no *dataset* completo 70% das amostras são de *emails* legítimos e 30% são *spams*, os 3 subconjuntos devem manter estas porcentagens.
- Em geral, mas não é regra, divide-se o *dataset* original da seguinte forma:
 - 70%, 15%, 15%
 - 80%, 10%, 10%

Divisão do *dataset*

- O conjunto de **treinamento** é para **treinar** os modelos de ML.
 - É onde o modelo “aprende” os padrões dos dados.
 - Quanto maior, mais diverso e variado, melhor o aprendizado.
- O conjunto de **validação** **avalia o modelo durante o desenvolvimento**.
 - Evita que o modelo decore os dados de treinamento (i.e., *overfitting*).
 - Ajustar hiperparâmetros.
 - Escolher arquiteturas.
 - Decidir quando parar o treinamento (*early stopping*).
- O conjunto de **teste** **valida a capacidade de generalização** dos modelos.
 - Usado uma única vez, ao final de todo o processo.
 - Fornece uma estimativa imparcial do desempenho real do modelo.
 - Simula dados nunca vistos.

Divisão do *dataset*

```
from sklearn.model_selection import train_test_split

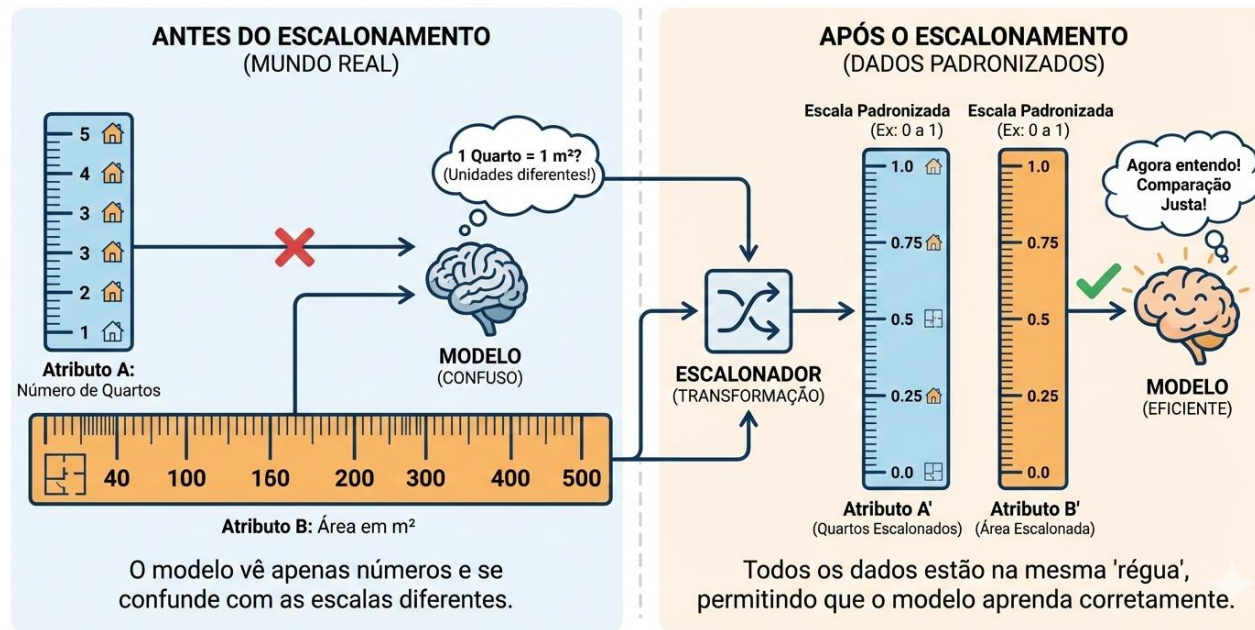
# X, y: conjunto total de dados, incluindo atributos e
# rótulos

# test_size: especifica a porcentagem de teste.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```



Agora veremos algumas **transformações** aplicadas aos dados que **ajudam no aprendizado dos modelos**, sendo a primeira delas, o **escalonamento**.

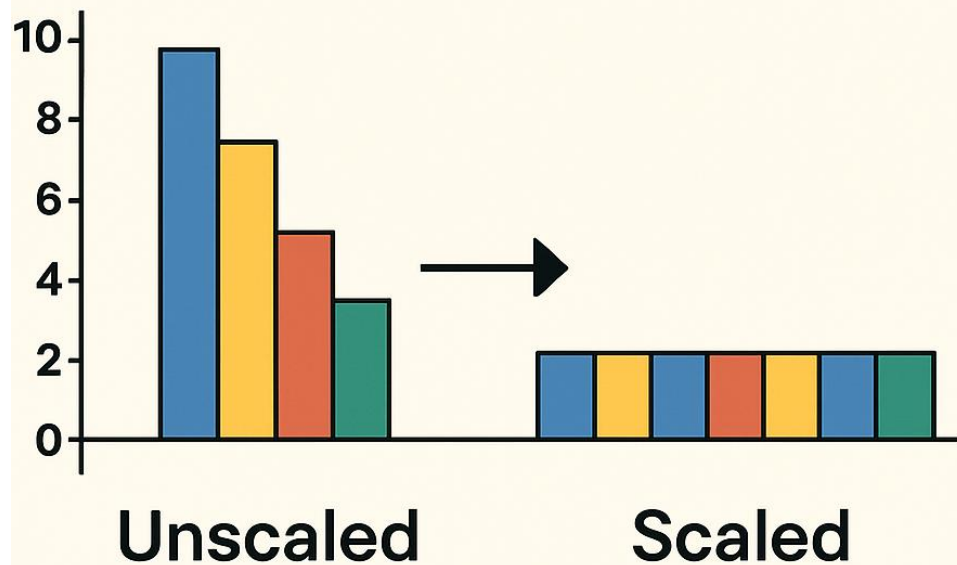
Escalonamento de atributos



- Precisamos colocar todos os dados na mesma régua (i.e., **escala**).
- Imaginem criar um modelo para a previsão do preço de casas.
 - **Atributo A:** Número de quartos (1 a 5).
 - **Atributo B:** Área em m^2 (40 a 500).
- Para o modelo, 1 quarto é igual a 1 m^2 .
 - Os modelos **não conhecem as unidades** do mundo real, eles **conhecem apenas números!**

Por que escalonar os atributos?

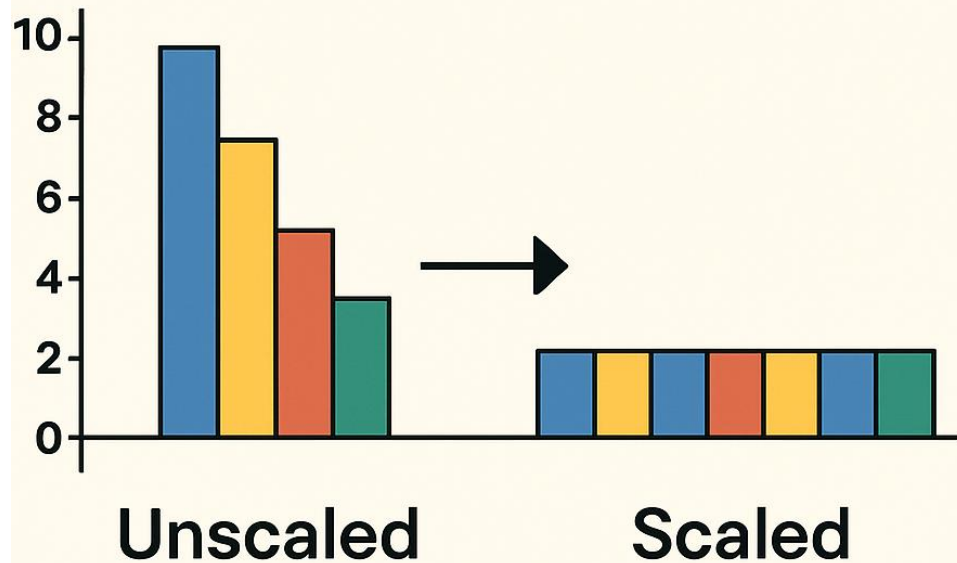
FEATURE SCALING



- Modelos de ML baseados em **distância** (e.g., KNN, SVM, PCA) ou **gradiente** (e.g., regressão linear/logística, redes neurais) **são sensíveis à escala dos atributos**.
- Atributos com **escala muito maior influenciam muito mais a previsão** dos modelos do que atributos com escala menor.
 - No exemplo da previsão do preço de casas, o modelo ignoraria os quartos e focaria só na área.

Por que escalonar os atributos?

FEATURE SCALING



- Mas e se todos os atributos são importantes para a previsão do modelo?
- O **escalonamento iguala a influência** dos atributos no modelo.
- Escalonamento transforma valores de atributos para uma mesma escala.
- Principais técnicas de escalonamento:
 - Normalização
 - Padronização
 - Escalonamento robusto

Normalização (*Min-Max Scaling*)

- **Como funciona:** Transforma os valores numéricos para uma **escala fixa, geralmente, entre 0 e 1**.
- **Fórmula:**

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \in [0, 1],$$

onde x é o valor original, x_{min} e x_{max} são os valores mínimo e máximo de um atributo (e.g., a coluna de um *dataset*).

- **Quando usar:**
 - Indicada para conjuntos de dados sem *outliers* (analisar e remover antes),
 - Com algoritmos que **não pressupõem** nenhuma **distribuição** (e.g., redes neurais, KNN),
 - Com algoritmos **baseados em distância** (e.g., KNN, SVM, K-Means) e **gradiente** (e.g., Redes Neurais, Regressor linear).

Normalização (*Min-Max Scaling*)



- **Cuidado:** Técnica muito sensível a *outliers*.
 - Um *outlier* extremo “**esmaga**” todos os outros valores **para um intervalo muito pequeno próximo de zero**.
 - Perde-se a **distinção entre valores típicos (i.e., *inliers*)**.
 - Isso reduz a eficácia do aprendizado, pois a **diferença entre os valores típicos é praticamente eliminada** na escala normalizada.

Normalização (*Min-Max Scaling*)

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Instância do normalizador com intervalo de 0 a 1  
scaler = MinMaxScaler(feature_range=(0, 1))
```

```
# Aprende os parâmetros de normalização (min e max) no  
conjunto de treinamento
```

```
X_scaled = scaler.fit_transform(X_train)
```

```
# Aplica o scaler aprendido ao conjunto de teste
```

```
X_test_scaled = scaler.transform(X_test)
```

Por que não calculamos os
parâmetros de
escalonamento usando o
conjunto completo?

Para não haver **vazamento** de informações do “futuro” no treinamento.

Data leakage

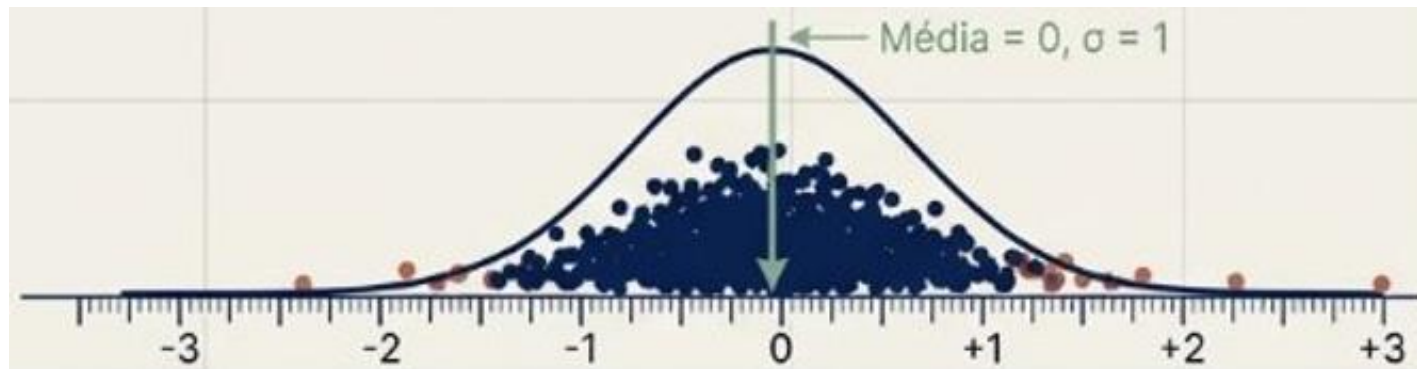
- *Data leakage* ocorre quando informações **que não deveriam estar disponíveis durante o treinamento** influenciam o **aprendizado do modelo, sua otimização ou sua avaliação**.
- O princípio é: **os conjuntos de validação e teste não devem influenciar as decisões de construção do modelo**.
- Os **parâmetros de pré-processamento devem ser aprendidos exclusivamente com os dados de treinamento** e posteriormente aplicados aos dados de validação e teste.
- Isso se aplica a praticamente todas as transformações que aprendem informações dos dados: escalonamento, imputação, seleção e extração de atributos, etc.

Padronização (*Standard Scaling*)

- **Como funciona:** Faz com que os atributos tenham **média 0 e desvio padrão igual a 1, não impondo limites.**
- **Fórmula:**

$$x' = \frac{x - \mu_x}{\sigma_x},$$

onde x é o valor original, μ_x e σ_x são a média e o desvio padrão de um atributo.



Padronização (*Standard Scaling*)

- **Vantagem:**

- Coloca os atributos em uma escala comparável, **expressando os valores em termos de desvios padrão em relação à média.**

- **Quando usar:**

- Em geral, com algoritmos que **assumem distribuições aproximadamente Gaussianas**
- ou **que dependem de operações baseadas em distância ou gradiente**, como regressor linear, SVM e redes neurais.

- **Cuidado:**

- Sensível a *outliers*, pois a **média e o desvio padrão** são **influenciados por valores extremos.**
 - Pode fazer com que os valores **típicos** sejam comprimidos próximos de zero.

Padronização (*Standard Scaling*)

```
from sklearn.preprocessing import StandardScaler
```

```
# Instância do padronizador
```

```
scaler = StandardScaler()
```

```
# Aprende os parâmetros de padronização no conjunto de  
treinamento
```

```
X_scaled = scaler.fit_transform(X_train)
```

```
# Aplica o scaler aprendido no teste
```

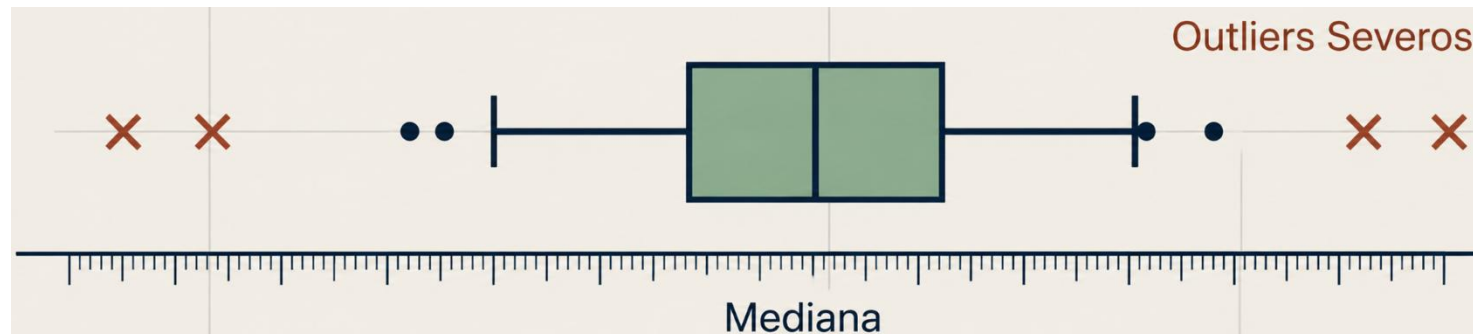
```
X_test_scaled = scaler.transform(X_test)
```

Escalonamento robusto (*Robust Scaling*)

- **Como funciona:** Usa a **mediana, Q_2 , e a distância interquartil (IQR)** no escalonamento.
- **Fórmula:**

$$x' = \frac{x - Q_2(x)}{\underbrace{Q_3(x) - Q_1(x)}_{\text{IQR}(x)}},$$

onde x é o valor original, $Q_2(x)$ e $\text{IQR}(x)$ são a mediana e a distância interquartil de um atributo.



Escalonamento robusto (*Robust Scaling*)

- **Vantagem:** Por utilizar a mediana e o IQR, **outliers têm influência reduzida** no escalonamento.
 - A mediana e o IQR são muito **menos afetados por poucos outliers extremos**.
 - Reduz o efeito de *outliers* extremos sobre a escala dos valores típicos, **evitando sua compressão excessiva em torno de zero**.
- **Quando usar:**
 - Quando há *outliers* significativos e desejamos mantê-los sem impactar o escalonamento.
 - Quando as distribuições são fortemente assimétricas (i.e., com cauda longa).
- **Cuidado:**
 - **Preserva melhor a escala** dos dados típicos, **mas não elimina nem disfarça** os *outliers*. Eles continuam nos dados.
 - Como os *outliers* não são removidos, algoritmos **baseados em distância** (KNN, k-means) e **gradiente** (RNA e Regressor linear) podem ter seu **aprendizado impactado**.

Escalonamento robusto (*Robust Scaling*)

```
from sklearn.preprocessing import RobustScaler

# Instância do escalonador robusto
scaler = RobustScaler()

# Aprende os parâmetros de escalonamento no treino
X_scaled = scaler.fit_transform(X_train)

# Aplica o scaler aprendido no teste
X_test_scaled = scaler.transform(X_test)
```

Armadilhas comuns durante o escalonamento

- Para evitar **vazamento de dados**, o escalonador deve sempre ser **ajustado no conjunto de treinamento**.
 - Assim, os conjuntos de validação e teste, são **transformados com os parâmetros encontrados com o conjunto de treinamento**.
- Geralmente, **não se aplica escalonamento à variável alvo**, i.e., os rótulos, apenas aos atributos.
- Em geral, **atributos binários ou one-hot não precisam ser escalonados**, especialmente quando já estão na mesma escala entre 0 e 1.

Codificação de atributos categóricos

- **Motivação:** Modelos de ML não entendem atributos categóricos (e.g., texto), portanto, precisamos convertê-los em números.
- As principais técnicas de codificação são:
 - One-Hot encoding
 - Label encoding
 - Target encoding
 - Binary encoding
- A escolha da técnica correta é fundamental, pois uma codificação inadequada pode induzir o modelo a interpretar falsas relações matemáticas.

One-Hot encoding

Atributo original

Color
Red
Red
Yellow
Green
Yellow



Atributo codificado

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

- Usada com atributos **nominais** (sem ordem/hierarquia) e com **baixa cardinalidade** (com poucas categorias distintas).
- **Como funciona:** Cria uma nova **coluna binária** para **cada categoria** distinta presente no atributo original, **indicando a presença ou ausência daquela categoria** em cada observação **de forma binária** (1 ou 0).

One-Hot encoding

Atributo original

Color
Red
Red
Yellow
Green
Yellow



Atributo codificado

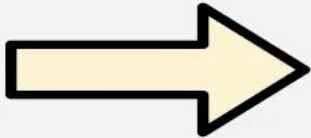
Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

- **Vantagem:** Evita que o algoritmo **assuma qualquer relação de grandeza ou ordem artificial** entre as categorias.
- **Limitação:** Gera **alta dimensionalidade** (muitas colunas) se um atributo possuir muitas categorias distintas.
 - Pode criar o problema conhecido como a “**maldição da dimensionalidade**”.
 - Pontos ficam **muito distantes uns dos outros** (perde-se noção de proximidade), prejudicando algoritmos baseados em distância (e.g., KNN).
 - Dificulta a visualização e aumenta o custo computacional.

Label e ordinal encoding

Atributo original

Color
Red
Green
Blue
Red



Atributo codificado

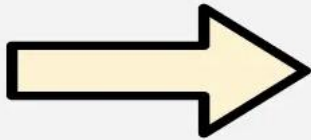
Color
0
1
2
0

- Converte **cada categoria de um atributo em um número inteiro único**.
- *Label Encoding* é usado na **conversão da variável alvo** (rótulos).
 - Por exemplo, animais: Gato=0, Cachorro=1.
 - Não existe necessariamente uma ordem: Cachorro não é maior do que Gato.
- *Ordinal Encoding* é usado na **conversão atributos** que possuem **ordem ou hierarquia**:
 - Por exemplo, tamanho de roupas: P=0, M=1, G=2.
 - Existe uma **ordem real**: $P < M < G$.

Label e ordinal encoding

Atributo original

Color
Red
Green
Blue
Red



Atributo codificado

Color
0
1
2
0

- Quando usar: **apenas com rótulos** (*label*) ou quando o **atributo possuir ordem ou hierarquia** (*ordinal*).
 - Caso contrário, usar *one-hot encoding*.
- **Risco:** *Ordinal encoding* pode introduzir uma **relação de ordem falsa**, prejudicando **modelos que interpretam os códigos como valores numéricos** (e.g., regressor linear, KNN e k-Means).
 - Por exemplo, na figura, **não existe uma relação de ordem entre as cores**: $\text{Red} < \text{Green} < \text{Blue}$.

Target encoding

Atributo original	Rótulo		Atributo original	Média do rótulo		Atributo codificado
workclass	target		workclass	target mean		workclass
State-gov	0		State-gov	0		0
Self-emp-not-inc	1	→	Self-emp-not-inc	1		1
Private	0		Private	1/3	→	1/3
Private	0					1/3
Private	1					1/3

- Em vez de usar números arbitrários (**Label/Ordinal Encoding**) ou criar várias colunas (**One-Hot Encoding**), **usa informação estatística extraída da variável alvo (i.e., rótulo)**.
- Substitui **cada categoria de um atributo** pela **média do valor do rótulo** correspondente àquela categoria.
- **Quando usar:** Útil para atributos de **alta cardinalidade**, pois não aumenta a dimensionalidade do *dataset*.

Target encoding

Atributo original	Rótulo		Atributo original	Média do rótulo		Atributo codificado
workclass	target		workclass	target mean		workclass
State-gov	0		State-gov	0		0
Self-emp-not-inc	1	→	Self-emp-not-inc	1		1
Private	0		Private	1/3	→	1/3
Private	0					1/3
Private	1					1/3

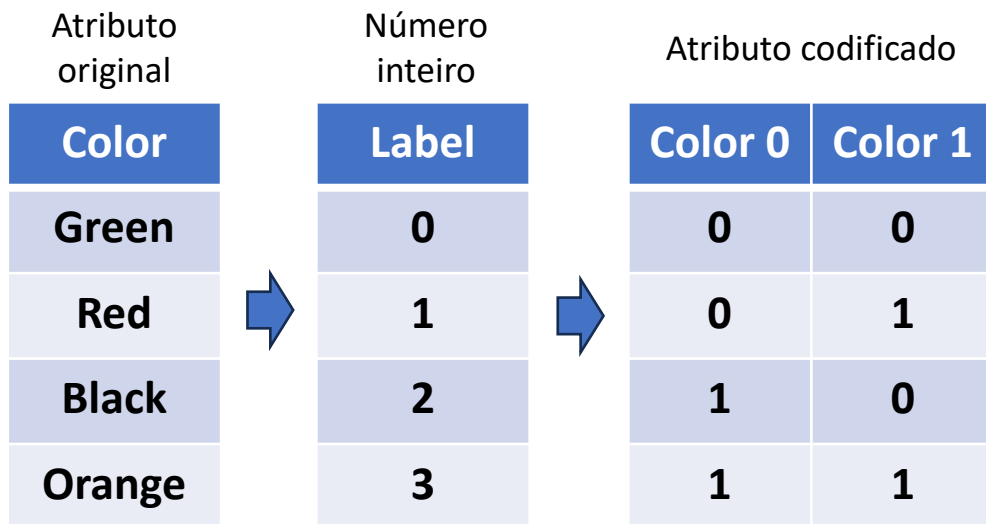
- **Risco:** *data leakage* e *overfitting*, especialmente em categorias com poucas amostras.
 - **Data leakage:** ocorre se os **rótulos do conjunto de validação/teste forem utilizados na codificação**.
 - Usar apenas conjunto de treinamento.
 - **Categorias raras:** poucas observações podem produzir **estimativas de média instáveis** (i.e., ruidosas).

Binary encoding

Atributo original		Número inteiro		Atributo codificado	
Color		Label		Color 0	Color 1
Green		0		0	0
Red	→	1	→	0	1
Black		2		1	0
Orange		3		1	1

- Cada **categoria** recebe um **número inteiro** (*Label*) que é **convertido para sua representação binária**, criando colunas para cada *bit*.
- **Quando usar:** Útil para atributos de **alta cardinalidade**.
- O número de colunas adicionais **cresce de forma logarítmica**, em vez de crescer linearmente como no *One-Hot Encoding*.
 - Por exemplo, um atributo com 4 categorias, exigiria 2 colunas ao invés de 4 com o *one-hot*.

Binary encoding



- **Vantagem:** **Reduz a dimensionalidade** em atributos de alta cardinalidade, **diminuindo o custo computacional** e **reduzindo problemas associados à alta dimensionalidade**.
- **Desvantagens:**
 - O **mapeamento** da categoria em um inteiro **é arbitrário** e pode **afetar o desempenho** de alguns modelos.
 - Como as categorias são representadas por **bits compartilhados**, **modelos lineares podem não representar** adequadamente **relações arbitrárias entre categorias**.
 - Modelos baseados em árvores lidam bem com isso.

Exemplos de *encoding*

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder,  
OrdinalEncoder, TargetEncoder
```

```
# SciKit-Learn não implementa o codificador binário (pip install  
category_encoders)
```

```
from category_encoders import BinaryEncoder
```

```
# LabelEncoder(), LabelEncoder(), TargetEncoder() ou BinaryEncoder()
```

```
enc = OneHotEncoder()
```

```
enc.fit_transform(X_train)
```

```
enc.transform(X_test)
```

```
enc.inverse_transform(X_test)
```

Engenharia de atributos

- **Definição:** **Processo de criar atributos relevantes** (mais úteis e informativos) a **partir dos existentes** para melhorar o desempenho de um modelo de ML.
- **Por que fazer?** Ajuda o modelo a **enxergar relações que não estão óbvias**.
- **Objetivo:** **Melhorar** a capacidade de **generalização do modelo**.
- **Exemplos:**
 - Data de nascimento → Idade, faixa etária.
 - Coordenadas GPS → Distância até certo ponto.
 - Preço e quantidade → Valor total da venda.
 - *Timestamp* → Hora, dia da semana, feriado?
 - Comprimento e largura → Área.
 - Atributos → Interação entre atributos: Polinômios (e.g., $x'_1 = x_1x_2 + x_1^2x_2$).
 - A codificação de atributos é considerada uma forma de engenharia de atributos.

Engenharia de atributos

```
import pandas as pd
```

```
# Exemplo de dados
```

```
df = pd.DataFrame({  
    "preco": [10, 20, 15],  
    "quantidade": [2, 1, 3],  
    "desconto": [0, 5, 0]})
```

```
# Feature Engineering
```

```
df["valor_total"] = df["preco"] * df["quantidade"]  
df["houve_desconto"] = (df["desconto"] > 0).astype(int)
```

Utilizamos o raciocínio humano e o conhecimento de negócio para criar novas variáveis.

Extração de atributos

- **Definição:** Processo que **transforma os dados originais** em uma **representação numérica mais adequada ou eficiente** para um algoritmo de aprendizado.
- **Por que fazer? Para**
 - tornar o dado utilizável pelo algoritmo (e.g., transformar texto em números),
 - destacar informação relevante,
 - remover redundância e ruído,
 - melhorar o desempenho do modelo (dados mais informativos e menos ruidosos).
 - reduzir a dimensionalidade, mas **perde-se parte da informação** no processo.
 - reduzir os recursos computacionais necessários para armazenamento e treinamento,
 - acelerar o treinamento,

Técnicas comuns para extração de atributos

- **Dados tabulares:** Análise de componentes principais (PCA) e análise de componentes independentes (ICA).
 - Em geral, o objetivo é reduzir o número de atributos originais.
 - Criam novos **atributos não correlacionados** que **capturam as direções de máxima variância** (PCA) ou **atributos que são independentes** (ICA).
- **Texto:** *Bag of words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), Word Embeddings.*
 - **Transformam texto em representações numéricas**, variando da **frequência de palavras** até **vetores que capturam significado e contexto**.
 - OBS.: modelos de ML entendem apenas números.

Técnicas comuns para extração de atributos

- **Imagem:** *Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), Convolutional Neural Network (CNN).*
 - Transformam os *pixels* de uma imagem em representações numéricas que capturam padrões visuais, como bordas, texturas, cores e estruturas semânticas, para facilitar o aprendizado dos modelos.
- **Áudio e séries temporais:** *Mel-frequency cepstral coefficients (MFCCs), Transformada rápida de Fourier (FFT), Espectogramas, Transformada de Wavelet.*
 - Transformam os sinais em representações que capturam propriedades espectrais ou de tempo-frequência.

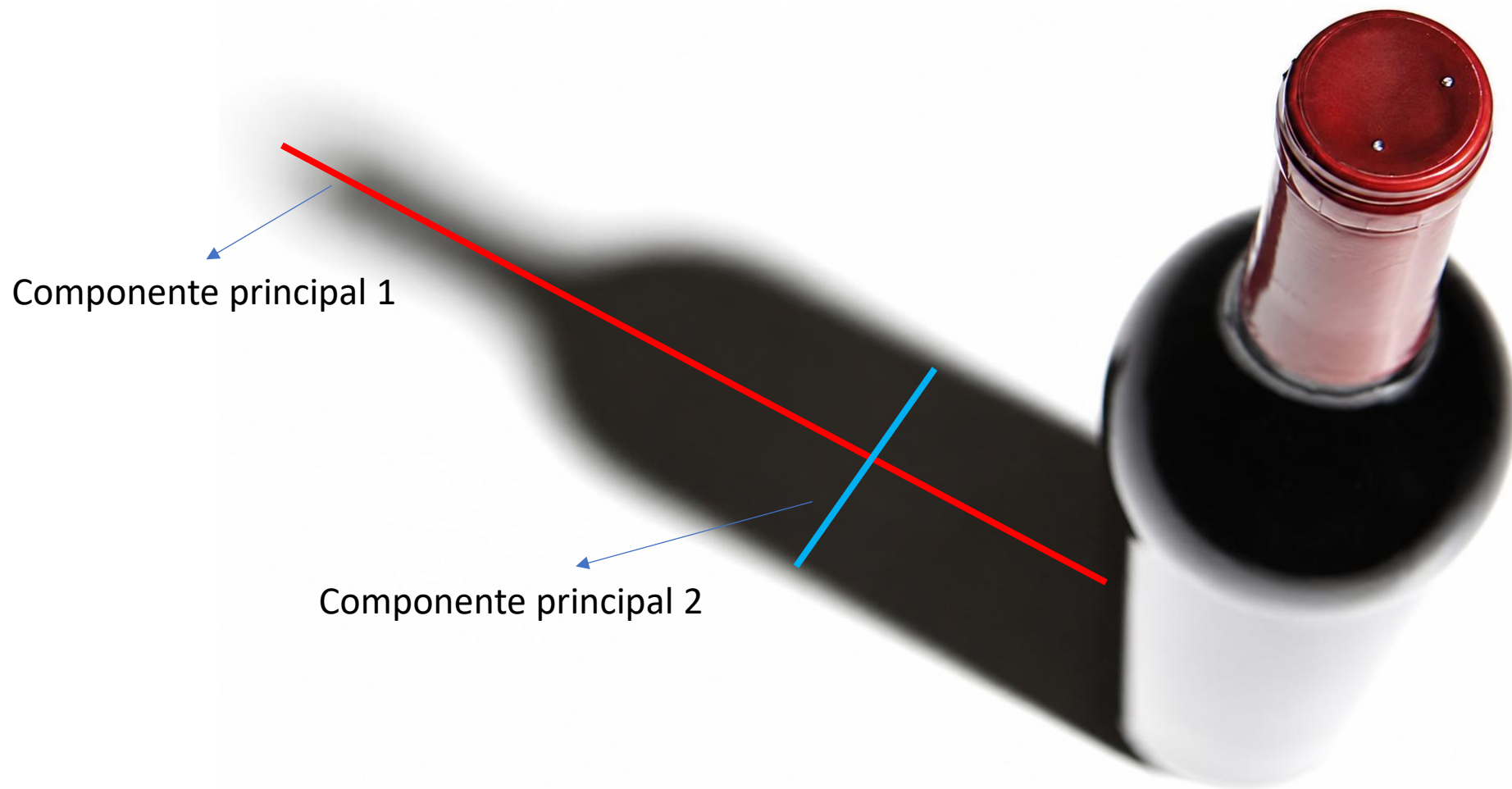
Redução de dimensionalidade

- Como trabalharemos bastante com dados tabulares, vamos entender uma das técnicas mais usadas para **redução de dimensionalidade** em dados estruturados (tabulares) a

Análise de componentes principais (*Principal Component Analysis - PCA*)



O PCA realiza uma **transformação linear** que **projeta** os dados originais **em um novo sistema de coordenadas**, definido pelas **direções que capturam a maior variância** presente nos dados.

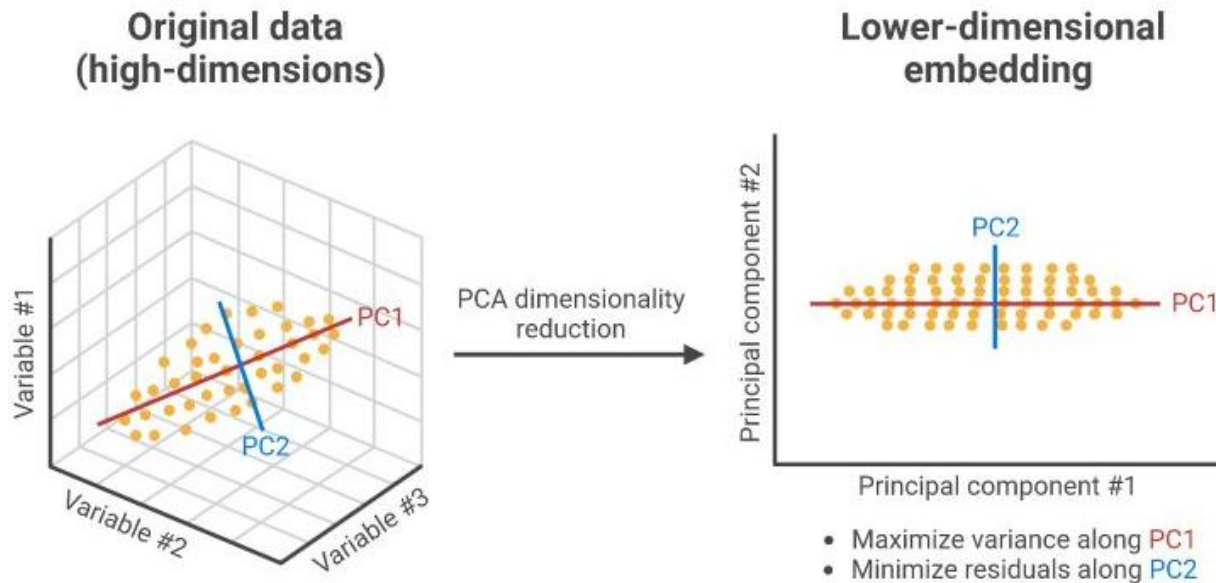


As **direções que capturam a maior variância** dos dados são chamadas de **componentes principais**.



O PCA pode **projetar** dados de **alta dimensão em um espaço de menor dimensão formado por algumas** componentes principais.

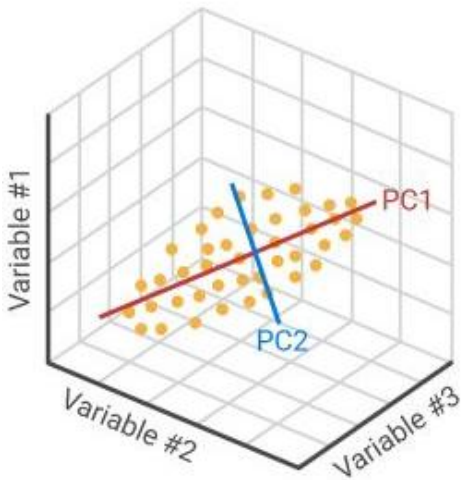
Análise de componentes principais (PCA)



- O PCA encontra as **direções onde os dados mais variam (componentes principais)** e **projeta** os dados nessas direções.
- Na figura ao lado:
 - PC1 (1ª Componente): é a **direção** que captura **a maior variância** presente nos dados originais.
 - PC2 (2ª Componente): é a **direção ortogonal** à PC1 que captura **a segunda maior variância** possível.
 - PC1 e PC2 contém grande parte da, se não, toda informação dos dados.

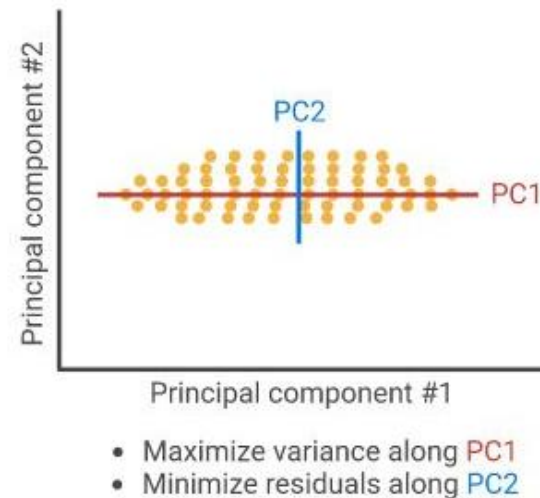
Análise de componentes principais (PCA)

Original data
(high-dimensions)



PCA dimensionality
reduction

Lower-dimensional
embedding



- As PCs são sempre **ordenadas da maior para a menor variância explicada**.
- Assim, é possível **representar os dados utilizando apenas as primeiras componentes**.
- Isso **preserva grande parte da informação original** enquanto reduzimos a dimensionalidade dos dados.

Análise de componentes principais (PCA)

- O PCA **assume relações lineares entre os atributos**.
 - Se os dados **não são lineares** (e.g., senoides, espirais, meias-luas e círculos), o **PCA falhará em preservar a estrutura dos dados**.
 - Nesses casos, podem ser utilizadas técnicas capazes de capturar estruturas não lineares, como Kernel PCA e Isomap.
- A **matriz de transformação deve ser calculada no conjunto de treinamento** e depois aplicada aos conjuntos de validação e teste.
- O PCA é **sensível à escala dos dados**.
 - Quando os **atributos estão em escalas diferentes**, **recomenda-se padronizá-los** antes da aplicação.
 - **Sem padronização, atributos com maior variância podem dominar** as componentes principais.

Análise de componentes principais (PCA)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
# Padronizar os dados antes de aplicar o PCA
```

```
X_train_scaled = StandardScaler().fit_transform(X_train)
```

```
X_test_scaled = StandardScaler().transform(X_test)
```

```
# Extrair 2 componentes
```

```
pca = PCA(n_components=2)
```

```
X_train_pca = pca.fit_transform(X_train_scaled)
```

```
X_test_pca = pca.transform(X_test_scaled)
```

```
print(pca.explained_variance_ratio_)
```

OBS.: “n_components” pode ser também um número entre 0 e 1 especificando o mínimo de variância explicada desejada.

Se não for definido, todas as componentes são usadas.

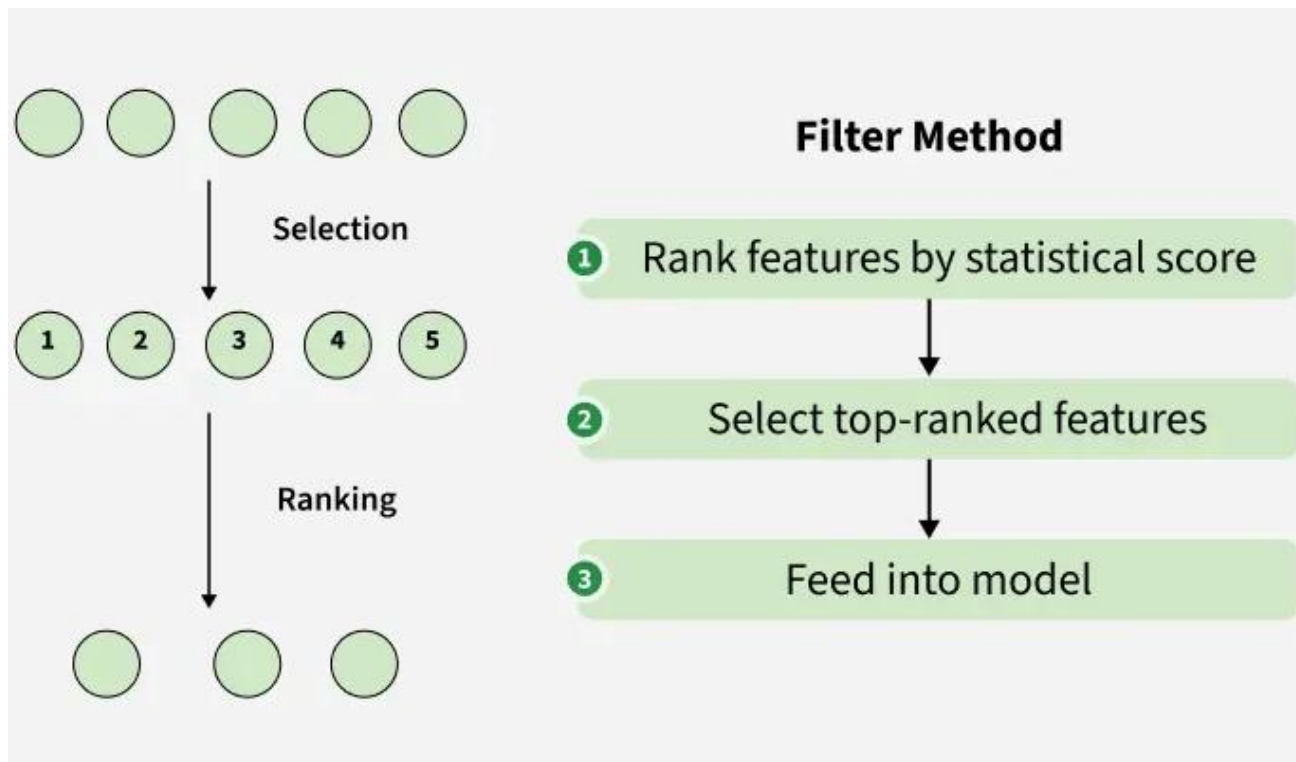
Seleção de atributos

- **Definição:** É o processo de **selecionar um subconjunto de atributos** que contribuem para a **melhoria do desempenho do modelo**.
- Por que usar?
 - **Reduz *overfitting*:** **Modelos mais simples** com menos atributos tendem a **generalizar melhor** para novos dados.
 - **Reduz custo computacional:** **Menos dados** significam **algoritmos** que processam **mais rápido**.
 - **Facilita interpretação:** É mais **fácil explicar** um modelo que utiliza 5 atributos do que um que utiliza 500.

Seleção de atributos

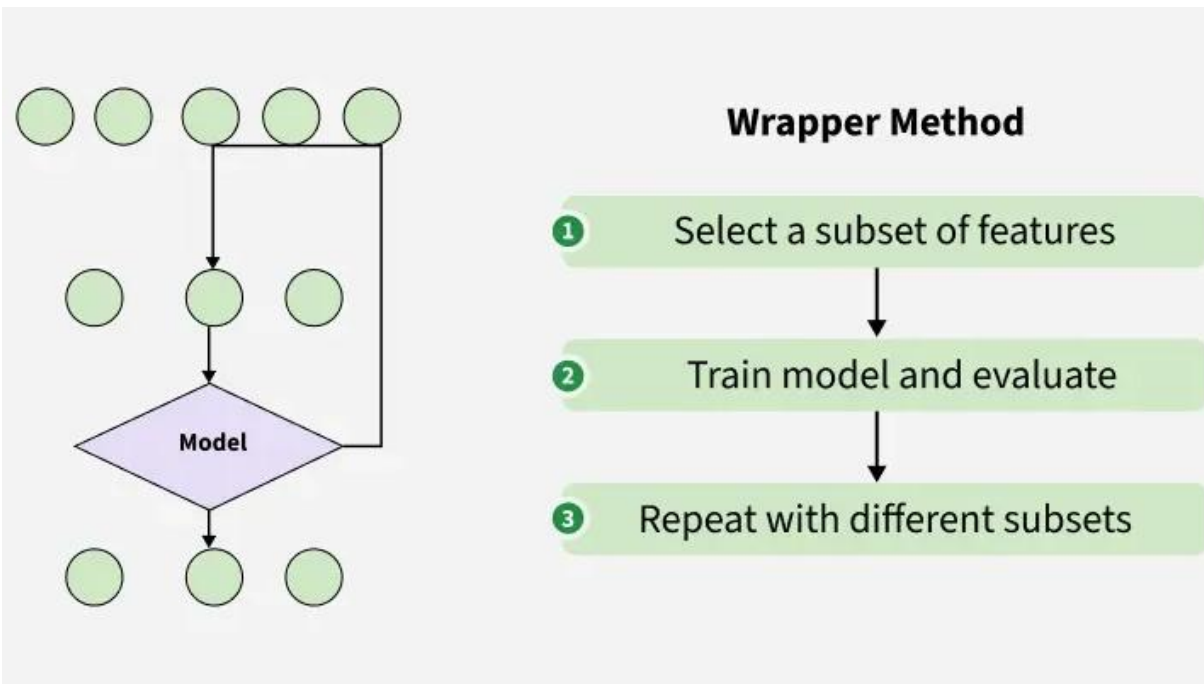
- Existem três tipos principais de técnicas de seleção de atributos:
 - **Filtragem:** Baseados em **medidas estatísticas dos dados**, como variância e correlação.
 - **Exemplos:** Correlação de Pearson/Spearman, limiar de variância, ganho de informação, informação mútua e teste qui-quadrado.
 - **Encapsulamento:** **Usa um modelo de ML para testar subconjuntos** de atributos e avaliar a performance.
 - **Exemplo:** **Eliminação recursiva** de atributos.
 - **Embarcado:** A **seleção ocorre durante o processo de treinamento** do modelo de ML. Essa técnica é **integrada** (i.e., embarcada) ao **treinamento do modelo**.
 - **Exemplos:** Regressão LASSO e árvores de decisão.
- **Filtragem usa apenas informações do *dataset***, já o **encapsulamento e o embarcado, usam informações do *dataset* e do modelo** para selecionar os atributos.

Métodos de filtro



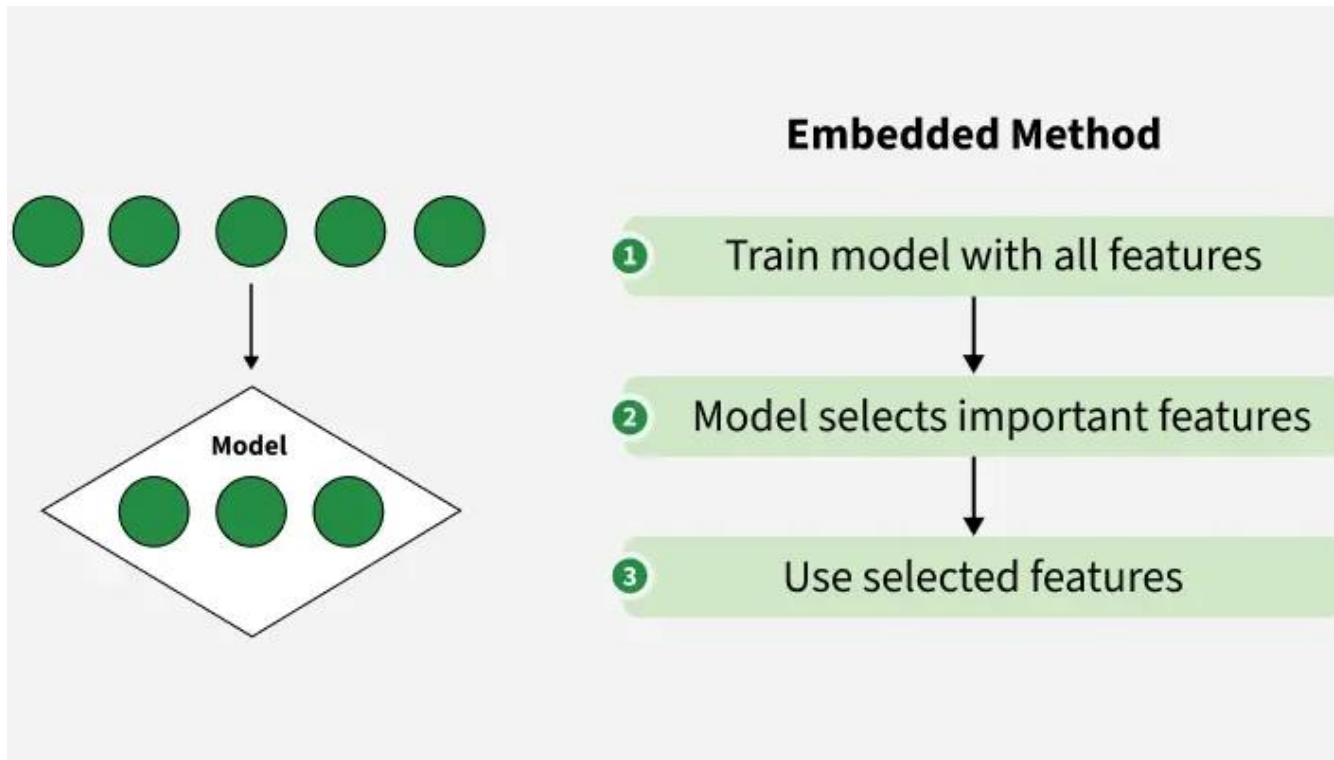
- São rápidos e independentes do modelo de ML.
- Escalam bem para *datasets* grandes.
- Filtros **avaliam os atributos de forma isolada, ignorando interações relevantes** entre eles.
- Porém, em muitos casos, o **valor preditivo de um atributo depende de sua interação com outros**.
- Nesses casos, usamos técnicas de encapsulamento.

Métodos de encapsulamento



- **Selecionam** o melhor subconjunto de atributos **usando um modelo de ML**.
- Em geral, têm **desempenho superior** aos métodos de **filtragem**.
 - **Consideram interações entre atributos** e avaliam o desempenho do modelo.
- Porém, são **computacionalmente mais caros**.
 - Envolvem o treinamento do modelo e, em alguns casos, busca iterativa.
- E são **suscetíveis ao overfitting**, pois podem **ajustar excessivamente** os **atributos** a um **modelo específico**.

Métodos embarcados



- Seleccionam atributos **durante o treinamento** do modelo.
- Apresenta **equilíbrio** entre **desempenho e tempo para seleção**.
- Porém, têm **interpretabilidade limitada** (por que um atributo foi removido?) e **não são suportados por todos os algoritmos**.

Seleção de atributos na biblioteca SciKit-learn

- O módulo `sklearn.feature_selection` implementa vários métodos de seleção de atributos, dentre eles podemos citar:
 - Filtragem
 - `VarianceThreshold`: Elimina atributos com variância abaixo do limiar configurado.
 - `SelectKBest`: Seleciona os k melhores atributos de acordo com um *score* definido (e.g., informação mútua, ganho de informação, etc.)
 - Encapsulamento
 - `Recursive Feature Elimination (RFE)`: Elimina os atributos de forma recursiva até que apenas os melhores permaneçam.
 - Embarcado
 - `SelectFromModel`: Seleção usando modelos com penalidade L1 ou baseados em árvores, como LASSO e árvores de decisão.

Seleção de atributos na biblioteca SciKit-learn

```
from sklearn.feature_selection import VarianceThreshold

# Remove atributos com variância <= 0.1
selector = VarianceThreshold(threshold=0.1)

X_train_reduced = selector.fit_transform(X_train)
X_test_reduced = selector.transform(X_test)
```

Extração vs. seleção de atributos

- A **extração transforma** os **atributos originais para criar novos**.
- A **seleção seleciona** um **subconjunto dos atributos existentes** mais relevantes **sem alterá-los**.

Exemplo

- [Exemplo: pre_processing.ipynb](#)



Lista de exercícios

Lista #4 – Transformações

Perguntas?

Obrigado!